Unit-1 Programming Languages

Er. Bhisma Raj Joshi
Department of Computer Engineering
[NAST]

@ER_BHISMARAJ

Objectives

- Introduction to Program, Software and Programming Language
- Analogy with Natural Language
- Classification of Computer Language(adv & dis-adv)
- Translator and It’s types (Assembler, Compiler and Interpreter)
- Linker and Loader
- Generation of Computer Languages
- Machine (Platform) Independent and Portability of Programs
- Some important types of High-Level Programming Languages

@ER_BHISMARAJ

Introduction to Program

- A program is a sequence or set of instructions written to perform a specific task or set of tasks by a computer or machine.
- Programs are always executed by computers or electronic devices, which enabling them to perform automated processes.
- Programs are usually written in programming languages (such as C, Python, C++, or Java) that a computer can interpret and execute.
- Programs are designed to solve problems, manage tasks, or process data efficiently for users or other systems.

@ER_BHISMARAJ

Introduction to Software

- Software is the intangible component of a computer system that consists of instructions, data, or programs that tell a computer what to do.
- It is the opposite of hardware, which refers to the physical components of a computer.
- Software provides a set of instructions that guide the computer's hardware to perform specific tasks. These instructions are organized into programs, which are designed to accomplish particular functions.
- **Examples:** Common software examples include Microsoft Word, Google Chrome, Windows, and Android.

@ER_BHISMARAJ

Characteristics of Good Software

- A well-designed software product should possess several key characteristics to ensure its effectiveness, efficiency, and user satisfaction. Here are some of the most important qualities:

- Correctness: The software should accurately fulfill its intended purpose and produce the expected outputs.
- Reliability: The software should be dependable and consistent in its performance, minimizing errors and crashes.
- Maintainability: The software should be easy to modify and update as requirements change, without introducing new defects.

@ER_BHISMARAJ

Portability:- The software can run on different platforms, operating systems, and hardware with minimal modification.

Customizability: It can be customized or tailored to meet specific user or business needs.

Scalability: The software should be able to handle increasing workloads or user numbers without compromising performance.

Security: The software should protect sensitive data and prevent unauthorized access.

@ER_BHISMARAJ

Assignment-1

Write down the differences between program and software.

@ER_BHISMARAJ

Introduction to Programming Language

- A programming language is a set of rules and symbols that are used to telling a computer to carry out a certain task.

- It serves as a bridge between humans and machines, enabling us to create software, applications, and websites.

- There are many different programming languages, each with its own unique syntax, semantics, and capabilities.

- Programming languages are also known as computer language.

@ER_BHISMARAJ

Each programming languages are divided into two components:-

Syntax

- Syntax refers to the rules governing the structure and arrangement of elements in a programming language. It's like the grammar of a language, how words, phrases, and sentences should be constructed.

Example:

- Incorrect syntax: printf("Hello world";) or Ram apple eats.
- Correct syntax: printf("Hello world"); Ram eats apple.

@ER_BHISMARAJ

Semantic

- Semantics refers to the meaning behind the syntactic elements of a programming language. While syntax defines how code should be written, semantics defines what the code does.
- In the context of every programming language, semantic checks correctness, means that the code performs the operations as intended by the programmer or not.
- Suppose a programmer wants to add 2 numbers

Example

sum = a - b; // Error in semantics: wrong operator or Ram eats house.

sum = a + b; // corrected version Ram eats apple.

- The codes are syntactically correct but logically wrong.

@ER_BHISMARAJ

Analogy With Natural Language

- Natural language is the language used by humans to communicate ideas, thoughts, and emotions with one another by spoken or written words.
- Programming languages and natural languages share several similarities, as both are systems of communication used to convey ideas and instructions. Below is an analogy between the two:

1. Syntax (Grammar)

- Each programming language has a specific syntax or set of rules that govern how code must be written.
- Similarly, natural languages like English, Spanish, or French have grammar rules (e.g., sentence structure, punctuation) that must be followed for communication to be clear.

@ER_BHISMARAJ

2. Semantics (Meaning)

- In programming language semantics refers to the meaning behind the code where as Semantics in natural language involves understanding the meaning of sentences and words. For example,
- The sentence "The cat is on the mat" has a clear meaning, describing a specific situation.

3. Keywords and Vocabulary

- Programming languages have reserved keywords, like if, else, for, and while, which cannot be used for other purposes. These are like the basic vocabulary of the language.
- Words like "and," "or," and "but" are also reserved in natural languages to construct meaningful sentences. Vocabulary expands as the language evolves, and words often have specific, predefined meanings.

@ER_BHISMARAJ

4. Compilers/Translators

- A compiler or interpreter translates code into machine language that the computer can execute.
- Humans "compile" or "interpret" natural language in their brains. When someone speaks or writes, their words are processed and understood based on learned grammar, context, and meaning.

5. Evolution

- New versions of programming languages are regularly released, introducing new features or simplifying syntax. Languages like Python, C++, and Java evolve with updates.
- Natural languages also evolve, with new words and slang entering common usage over time. For example, words like "selfie" or "emoji" are relatively new additions to English.

@ER_BHISMARAJ

Differences between Natural and Programming Language

Natural Language	Programming Language
- Natural Language has a very large vocabulary. - Natural language is easily understood by humans. - Natural language is ambiguous in nature. - It is used for human communication. - Evolves naturally over time, influenced by culture, social change. Example Hindi, English.	- Computer language has a very limited vocabulary. - Computer language is easily understood by the machines. - It is unambiguous. - Designed for computers to understand and execute instructions. - Evolved based on technology needs and influenced by the software industry. - Examples C, C++, Java etc.

@ER_BHISMARAJ

Types of Programming Language

- Computer languages can be categorized into two types, each serving different purposes and having unique characteristics. Here are the main types:

1. Low Level Language
2. High Level Language

1. LOW LEVEL LANGUAGE:-

- A low-level language is a type of programming language that is very close to the computer's hardware it means that the language directly interacts with the computer's physical components like the CPU and memory.

@ER_BHISMARAJ

- Low level language are also called machine dependent language which means program written for one type of system cannot be run in another system.
- Further low-level programming languages are divided into two types:-

i) Machine Level Language[1GL]:-

- Machine level language which is also known as machine code is the lowest- level programming language. It consists of binary digits (0 and 1) that the computer's CPU can execute directly.

- This language is specific to each computer's architecture and is used to perform basic operations like arithmetic, data movement and control instructions.

@ER_BHISMARAJ

Advantages(Merits)

- It is directly understood by the processor so has faster execution.
- Language translator is not needed.
- Requires less memory space as compared to high level programming languages.

Disadvantages(Demerits)

- It is very difficult to program using 0 and 1s.
- Use of this language makes programming time consuming.
- Difficult to find errors and to debug.

@ER_BHISMARAJ

ii) Assembly Language[2GL]

- It is also known as low level programming language. This language uses symbolic instructions in place of 0 and 1, which are called mnemonics.

- Examples includes ADD, SUB, JMP,RET, PRINT etc.

- The program written in assembly language is converted into machine code by assembler thus resulting program is called an object code.

@ER_BHISMARAJ

Advantages of Assembly level language

- Assembly language allows for highly optimized code that can run faster than code written in higher-level languages.
- Programs written in assembly language can be very compact, making efficient use of memory
- Since assembly language is closer to machine code, it doesn’t rely on compilers to optimize the code.

Disadvantages of Assembly level language

- Writing program in assembly language is complex and time consuming.
- Debugging and maintaining assembly code can be challenging.
- Assembly language is specific to a particular computer architecture.

@ER_BHISMARAJ

2)High Level Language

- A high-level language is a type of programming language designed to be more user-friendly and closer to natural language, making it easier for humans to read and write code.

- Unlike low-level languages that are closely aligned with a computer’s hardware, high-level languages use syntax and semantics that resemble human languages.

- It uses familiar English words, special characters and mathematical symbols in their syntax which makes easier to read, write and understand.

@ER_BHISMARAJ

Advantages of High Level Language

- Programmer friendly and easy to understand
- Portable and machine independent
- Easy to modify and maintain
- Easy to debug

Dis-advantages of High Level Language

- Need to be translation into machine code
- Slower execution time
- Less control over hardware
- High memory usage

@ER_BHISMARAJ

Assignment-2

Write down the major differences between low level and high-level programming language.

@ER_BHISMARAJ

Programming Language Translator

- Translator which is also called language processor is a software tool that converts a computer program written in one language to another language.
- It takes a program written in high level language which is called source code, and convert that source code into machine code which is directly understand by CPU.
- It also helps to find out the error during translation process.

@ER_BHISMARAJ

There are 3 different types of Translator

1. Assembler

- Assembly language is a low-level programming language that is closely related to machine code but is more readable for humans.

- An assembler is a type of software that translates assembly language code into machine code, which is the binary code that a computer’s processor can execute directly.

- Assemblers can be categorized into two types based on the number of passes they take to process the input.

@ER_BHISMARAJ

i) One Pass Assembler

- A single-pass assembler processes the assembly code in one pass. It scans through the entire program once and generates the machine code directly.
- This approach is faster.

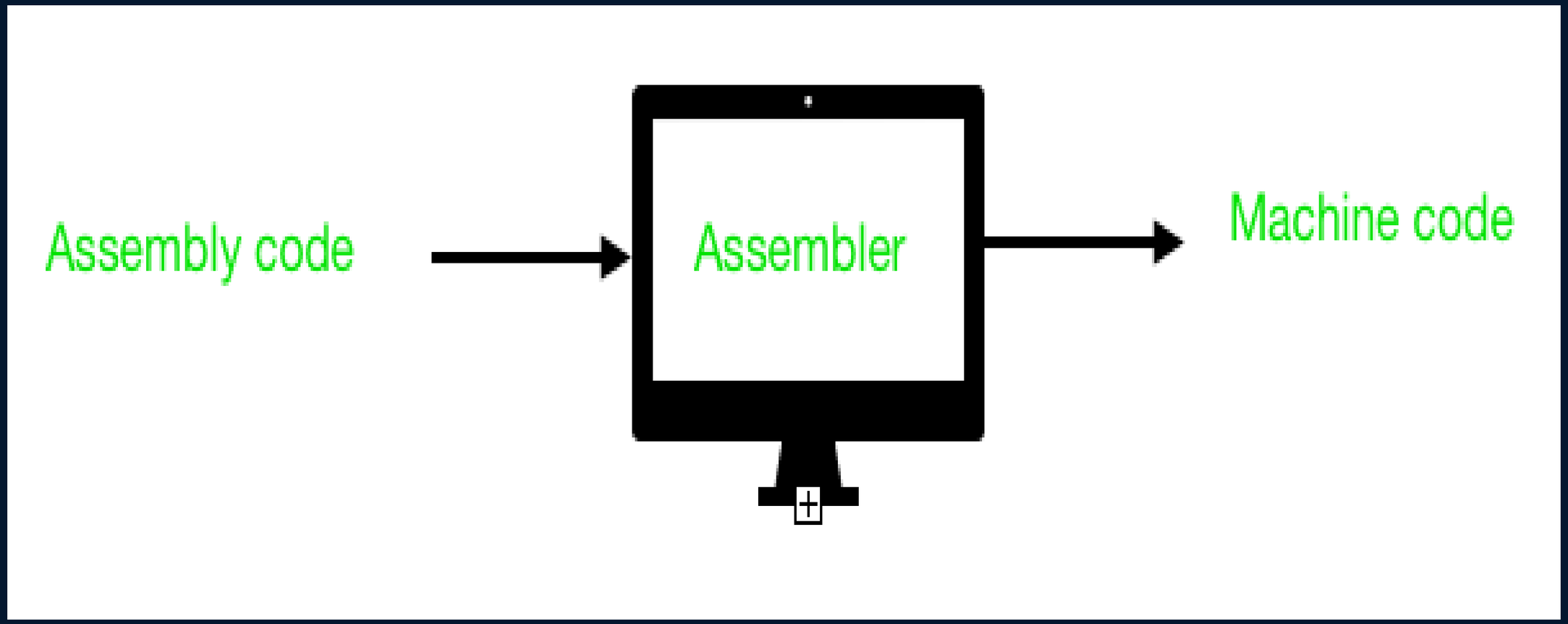
ii) Two Pass Assembler

- A two-pass assembler processes the source code in two passes:

• **First Pass:** The assembler scans the code to build a symbol table, which contains the addresses of all variables.

• **Second Pass:** In this pass, the assembler translates the code into machine language, using the symbol table for addresses of variables.

@ER_BHISMARAJ



@ER_BHISMARAJ

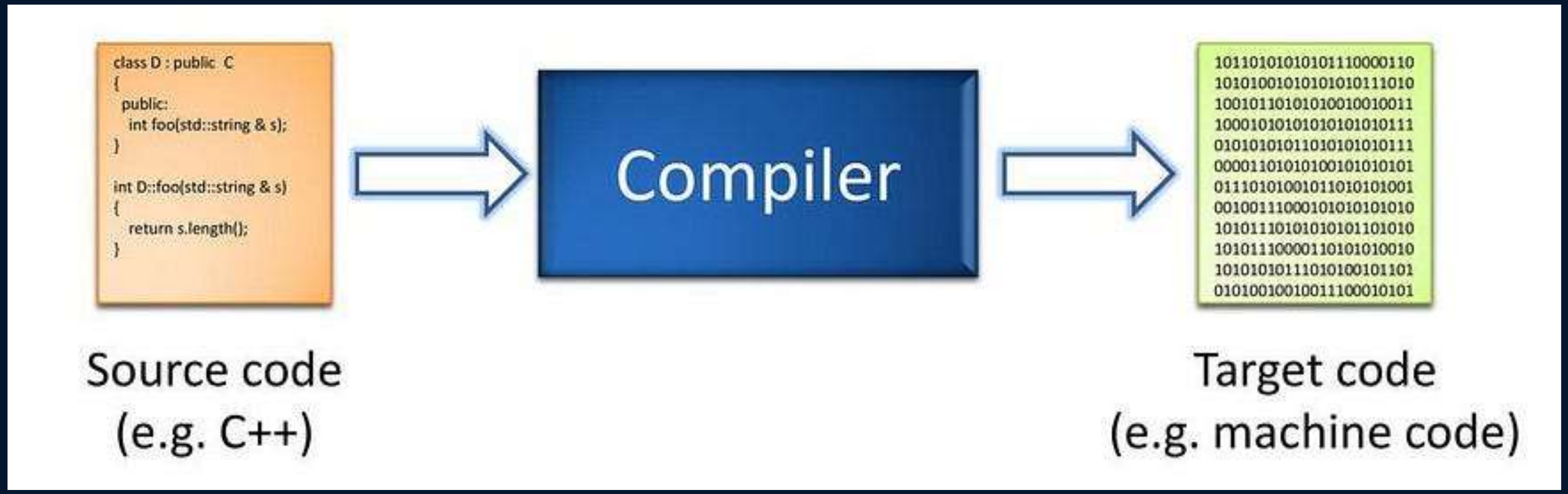
2. Compiler

- A **compiler** is a software tool that translates code written in a high-level programming language (such as C, C++, Java, or Python) into machine code or bytecode code which can be directly executed by a computer.

- This translator reads the complete program at first if it is bug free then it converts the source code into machine code at once. If it detects the errors then the compilation process gets terminated. Thus, it also helps to find out the errors.

@ER_BHISMARAJ

- Simply compiler is processor dependent or platform dependent means the codes only works for the system, so, to overcome such problem cross compiler can be used.



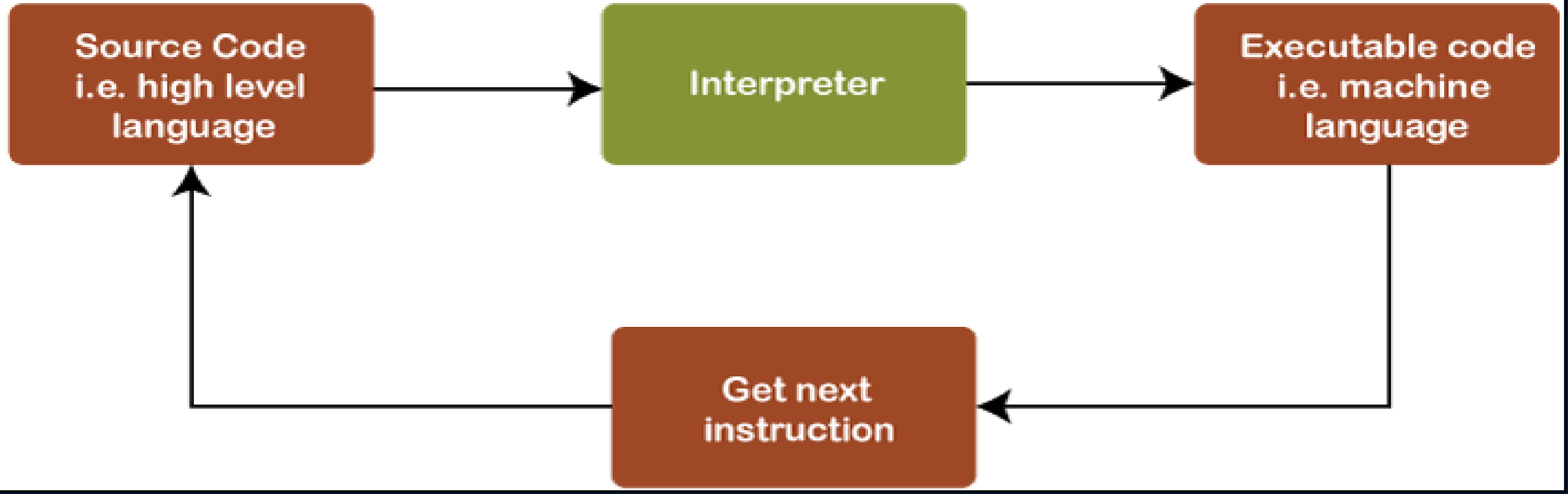
@ER_BHISMARAJ

Interpreter

- An interpreter is also a software program that translates a source code into a machine language.
- However, an interpreter converts high-level programming language into machine language line-by-line while interpreting and running the program.
- Interpreter reads only single statement of a program then translates it and execute it. After that it again read the next statement of the program, translate it and executes.

@ER_BHISMARAJ

How Interpreter Works



@ER_BHISMARAJ

Interpreter vs Compiler

Feature	Interpreter	Compiler
Translation method	Line-by-line during execution	Translates the entire program at once
Output	Executes code directly; no object code	Generates object code (machine code)
Execution speed	Generally slower (since translation happens at runtime)	Generally faster (once compiled)
Error handling	Stops at the first error; easy to debug	All errors are reported after compilation
Memory usage	Lower since no object code is produced	Higher due to object code and linked libraries
Development process	Faster, due to immediate feedback	Slower, due to the need to compile before testing
Example languages	Python, JavaScript, Ruby	C, C++, Rust, Go

@ER_BHISMARAJ

Assignment:-3

Why program translator is required? Explain it's various types in detail.

@ER_BHISMARAJ

Linker

- A **linker** is a program that takes one or more object files generated by a compiler and combines them into a single executable program.
- Object files are the intermediate outputs of the compilation process, containing machine code and related data.
- The linker is responsible for resolving references between various parts of a program and ensuring that the code is correctly combined and addresses are correctly assigned for function calls and variables.

@ER_BHISMARAJ

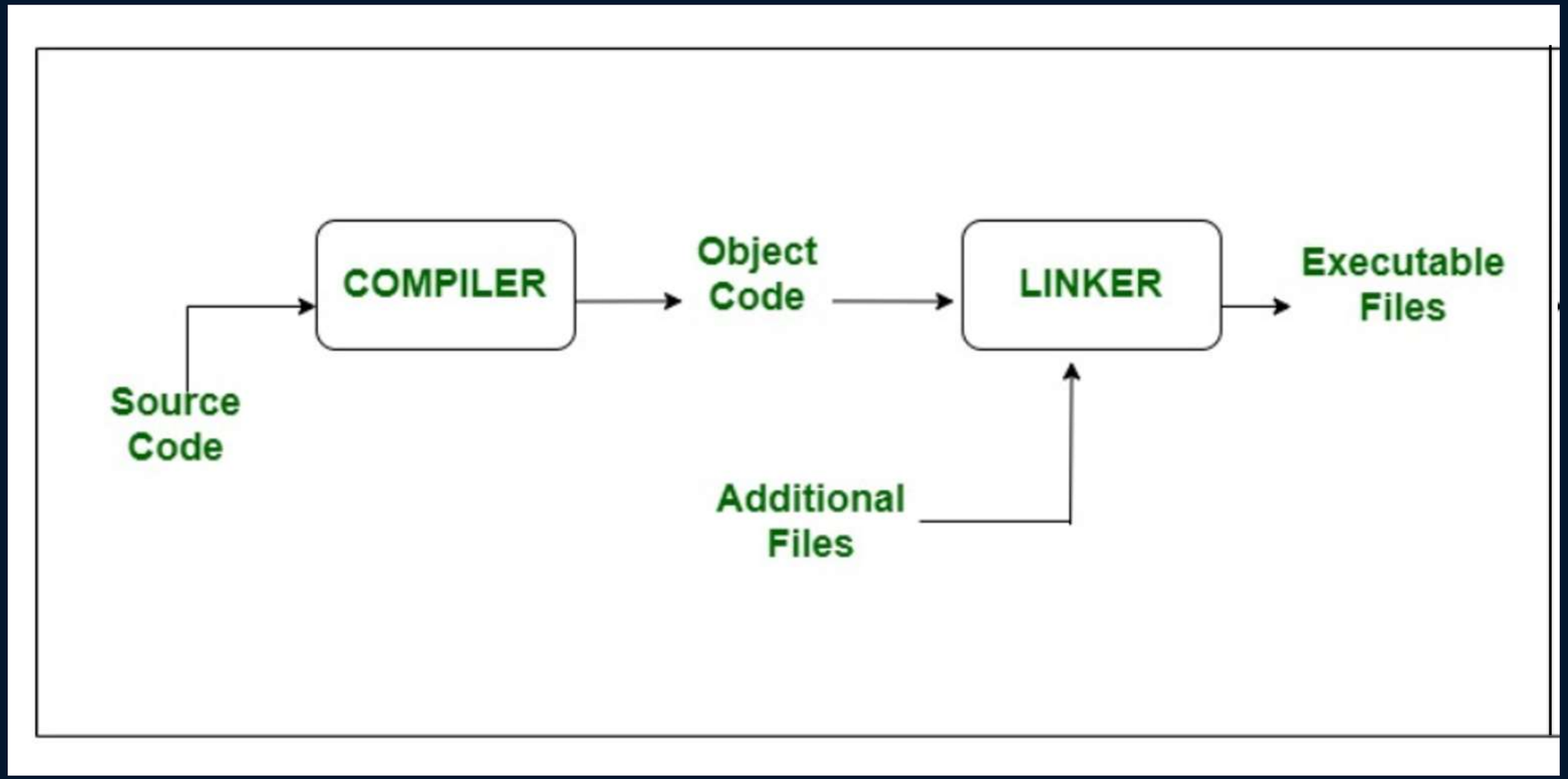


Fig:- Working mechanism of Linker

@ER_BHISMARAJ

Types of Linker

1. Static Linker

- A static linker is also known as the Compile time linker.
- It is the type of linker that combines the object files and other libraries into a single executable file at compile time.
- The resulting executable file contain all the necessary code and no further linking is required at runtime.
- Example: Linking in C/C++ where all functions from libraries are copied into the executable file.
- Larger executable size, since all the code (even unused functions) from linked libraries are included.

@ER_BHISMARAJ

2. Dynamic Linker

- It is also known as Runtime Linker.
- A dynamic linker links libraries file and other object files to an executable file at runtime rather than at compile time.
- This can reduce the size of the executable file and allow for updates to shared libraries without recompiling the entire program.
- Example: In Linux, the dynamic linker (ld.so) resolves references to shared libraries when the program starts.

@ER_BHISMARAJ

Loader

- A **loader** is a crucial component of an operating system responsible for loading executable programs and libraries into memory, preparing them for execution.
- It performs the following key tasks.

Loading:- It reads the executable file from disk and loads it into the main memory.

Relocation:- Adjusts the addresses used in the program

Preparation for Execution:- Sets up the necessary environment for the program to run.

@ER_BHISMARAJ

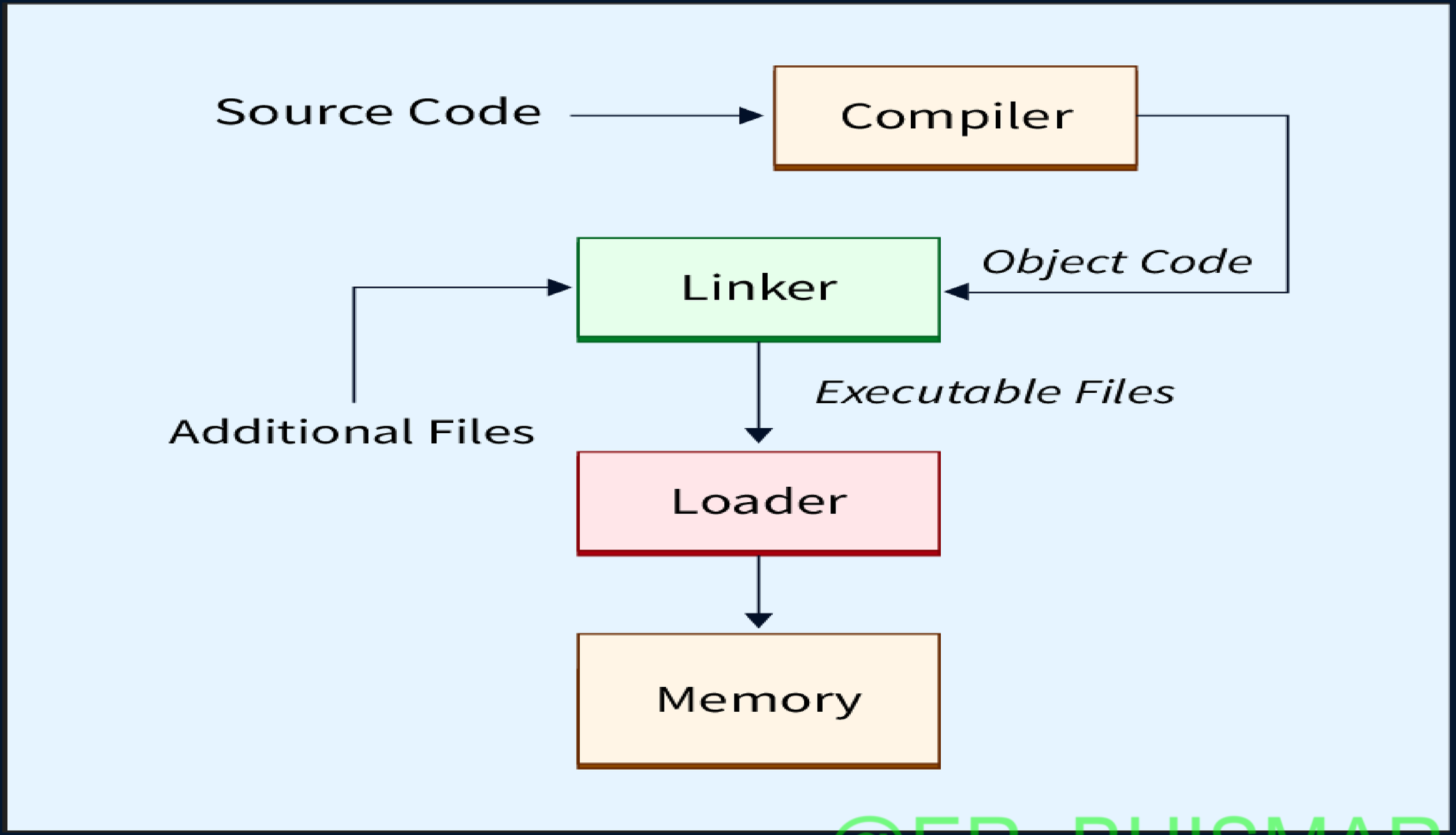


Fig:- Working mechanism of Loader

@ER_BHISMARAJ

Types of Loaders

1. Absolute Loader:

- An absolute loader directly loads the program into memory at a specific location defined by the program itself.
- The memory addresses in the program are absolute and do not change.
- The loading process is simple and fast.
- The program must be designed to run at specific memory addresses, making it inflexible, especially in multitasking environments.

@ER_BHISMARAJ

2. Relocating Loader

- A relocating loader is a type of loader that allows a program to be loaded into different regions of memory at different times.
- This flexibility is crucial for efficient memory utilization and multitasking in modern operating systems.
- Unlike an absolute loader, it doesn’t require the program to be loaded at a specific address.

@ER_BHISMARAJ

3. Compile and Go Loader

- A compile-and-go loader is a type of loader where the compilation and execution of a program happen in a single step.

- This approach is often used in simple systems or development environments where quick testing and execution are needed.

- The compile-and-go loader is fast because it eliminates the need for separate compilation, linking, and loading phases.

@ER_BHISMARAJ

Importance of Linker and Loader

Symbol Resolution: Linker resolves references between different parts of a program, connecting function calls and variable references.

Code Reusability and Efficiency: By linking reusable libraries, the linker and loader allow developers to avoid redundant code, enhancing efficiency.

Memory Management: The loader assigns memory locations to program components and handles any necessary relocations.

Dynamic Linking: The loader supports dynamic linking, loading libraries only when needed at runtime.

Execution Preparation: The loader sets up the environment required for program execution.

@ER_BHISMARAJ

Assignment

Explain the roles of linkers and loaders in the process of executing a program in a computer system. What are the differences between static and dynamic linking?

@ER_BHISMARAJ

Generation of Computer Language

- The term generation of computer language refers to the classification of programming languages based on their level of abstraction from machine code and their evolution over time.

- Five Generations of Computer Languages:

1. First-Generation Language (1GL): Machine Language
2. Secondary-Generation Language (2GL): Assembly Language
3. Third-Generation Language (3GL): High-Level Language
4. Fourth-Generation Language (4GL): Very High-Level Language
5. Fifth-Generation Language (5GL): AI Language

@ER_BHISMARAJ

3. Third Generation Language(3GL)

- A Third-Generation Language (3GL) is a high-level programming language.
- It follows the concept of structured programming and procedure-oriented language.
- It was designed to be more user-friendly and closer to human language, making programming easier and more productive.
- 3GLs require a compiler or interpreter to translate the high-level source code into machine code so it can be executed by the computer’s hardware.

@ER_BHISMARAJ

Advantages

- POP is straightforward and simple to implement, especially for small and medium-sized projects.
- It can be memory efficient since POP avoids the overhead of object-oriented structures like classes and objects.
- It is easier for beginners to learn.
- POP generally has less overhead compared to Object-Oriented Programming (OOP) because there are no classes or objects.

Disadvantages

- Difficulty in Managing Large Projects.
- Lack of Encapsulation
- Difficulty in Maintaining code.
- 3GLs typically consume more memory than lower-level languages.

@ER_BHISMARAJ

4. Fourth Generation Language(4GL)

- It is the refined version of third generation programming language.
- The aim of fourth generation programming language is to provide higher level of abstraction.
- Object oriented programming is the most essential feature that is provided by the fourth-generation programming language.
- They also include support for database management, report generation, GUI development web development etc.
- Examples JAVA, .NET, My SQL, PHP etc.

@ER_BHISMARAJ

Advantages

- 4GLs are more similar to natural language, making them easier for non-programmers.
- Reduced Development Time: because of their simplicity and abstraction.
- Machine independent.
- Easy for debugging.

Disadvantages

- Requires more memory space as compared to 3GL
- Requirement of hardware resources is high.
- Some 4GLs require specialized development tools or environment which may be costly to license or set up.
- 4GLs add layers of abstraction, the code may run slower.

@ER_BHISMARAJ

5. Fifth Generation Language(5GL)

- The decision-making language focus on problem-solving and abstract logic.
- These languages are often used for Artificial Intelligence application including expert system, reasoning and decision-making system.
- Some 5GLs enables interaction through natural language processing, allowing users to input commands or queries in a format similar to daily language.
- Example Prolog, LISP, Mercury etc.

@ER_BHISMARAJ

Advantages

- High level Abstraction:- which makes programming more closer to the natural language.
- High problem-solving capability :- Complex problem-solving tasks, such as artificial intelligence and automated reasoning.
- These languages are designed to represent complex knowledge structures (e.g., facts, rules, relationships) more naturally.

Disadvantages

- They are not for general purpose programming like web development, system software and application development.
- Slower than other programming due to high level of focus, logical reasoning.
- The logical reasoning, logical computation can still be difficult to learn and master.

@ER_BHISMARAJ

Machine(Platform) Independent and Portability of Programs

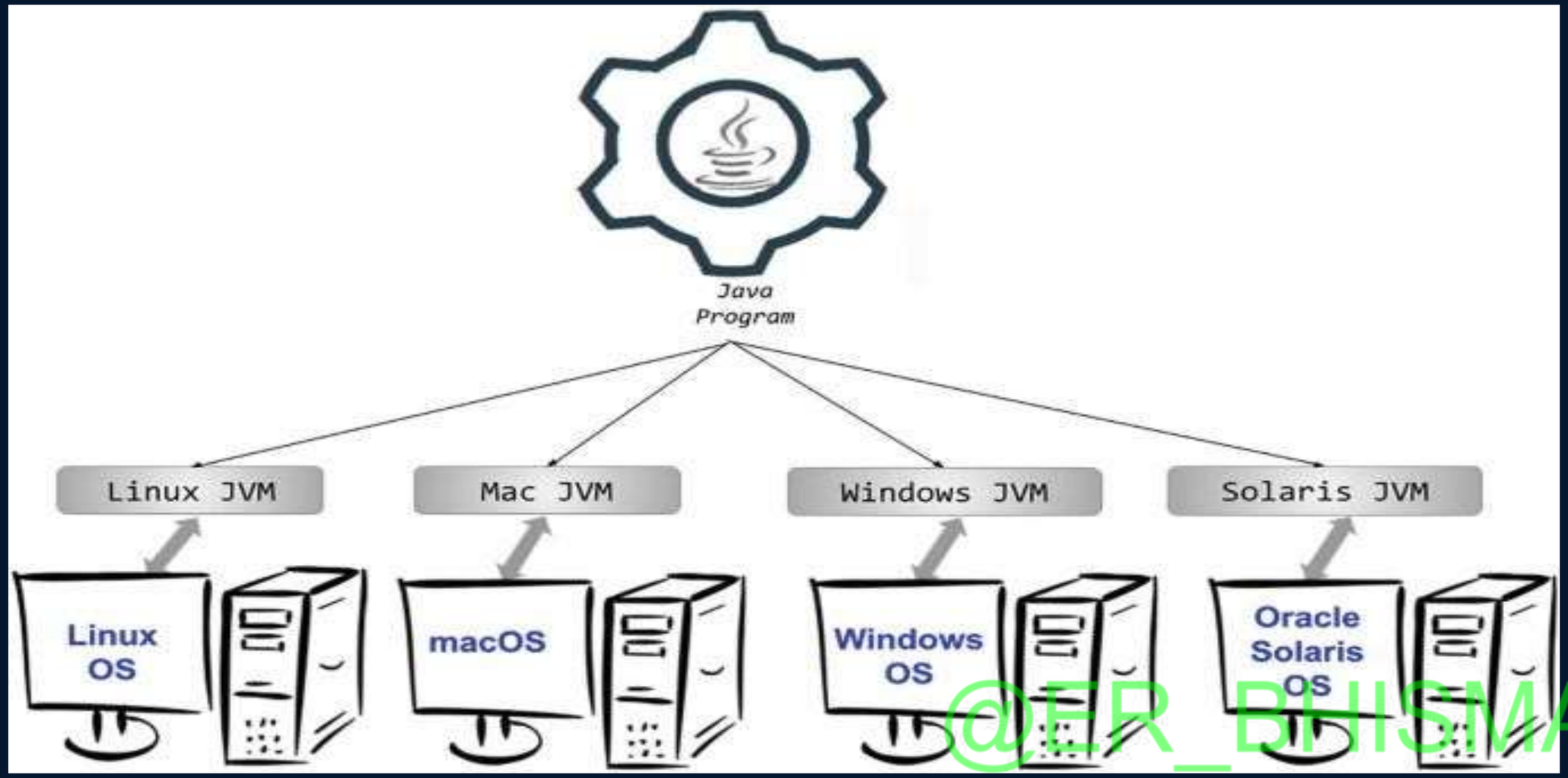
- Machine (Platform) Independence and Portability of Programs refer to the capability of software to run on different hardware platforms or operating systems without requiring significant modification.

- This is a crucial characteristic in modern software development as it ensures flexibility, reduces development costs, and broadens the reach of the application.

- A machine-independent program can run on various types of systems (hardware and operating systems). The developer does not have to rewrite the program for different environments.

@ER_BHISMARAJ

For example, in Java, the program is compiled into bytecode, which is executed by the Java Virtual Machine (JVM). The JVM is available for many platforms, making Java programs platform-independent.



@ER_BHISMARAJ

- Portability is closely related to platform independence, but it specifically refers to the ease with which software can be transferred from one environment to another.
- Portable software requires minimal changes to work in different environment making it easier to adapt to the new hardware or software platform.
- For example, python script can be run on multiple O.S (windows, mac os, Linux) with little to no modification making them portable.

@ER_BHISMARAJ

Benefits of Portability and Platform Independence

Cost Efficiency:- Reduces the need for ,multiple versions of the same software for different platforms, lowering development and maintenance costs.

Flexibility:- Easier to adapt to new technologies or migrate to new systems.

User Reach:- Expands the potential user base by making software available on a wide range of devices and operating system.

@ER_BHISMARAJ

Important types of high-level language

- As we know a high-level language is a type of programming language designed to be more user-friendly and closer to natural language, making it easier for humans to read and write code.
- Some important high-level languages are follows...

a) Python:- Python was created by Guido van Rossum in 1991 as a simple, versatile, and readable programming language.

- Python's simplicity and versatility make it a popular choice for a wide range of applications like web development, Data Science and Analytics, Machine Learning and AI etc.
- Real-life Example: Python is used by Google for various purposes, including back-end services and data analysis.

@FR_BHISMARAJ

b) Java:- Java was developed by **James Gosling** and his team at Sun Microsystems, released in 1995.

- Java is known for its portability across different platforms due to the Java Virtual Machine (JVM). It is used in various fields like android app development, Banking app web application, enterprise application etc.

c) JavaScript :- It was created by Brendan in 1995. It is widely used in web development enabling dynamic and interactive web applications.

- Frontend Development: JavaScript is used alongside HTML and CSS to build dynamic and responsive user interfaces.
- Backend Development: With Node.js, JavaScript can also be used on the server side, enabling full-stack development using a single language.

Example: Websites like Facebook, Twitter, and YouTube utilize JavaScript

@ER_BHISMARAJ

d) C++ :- It was developed by **Bjarne** at Bell Labs in 1983 as an extension of the C programming language. It is known for its performance and efficiency, making it suitable for system programming, game development and development of the window OS.

e) PHP:- PHP (Hypertext Preprocessor) was created by **Rasmus Lerdorf** in 1994 it is primarily used for server-side web development and embedded inside HTML, also used in content management system like WordPress, for accessing database like MySQL Oracle.

@ER_BHISMARAJ

f) Swift :- Swift was developed by **Chris Lattner** in 2010 for Apple Inc. Swift is the language of choice for Apple ecosystem development.

- It is used to build apps for iPhones, iPads, Apple Watches, and Apple TVs. It is also used for developing applications for Mac computers
- Almost all modern apps on the Apple App Store, including those from major companies like Uber and Airbnb, are developed using Swift.

@ER_BHISMARAJ